SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA0606

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull

problems (BL1, BL2,BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 1 **Total Marks:50**

Annexure A

DEBUGGING & OPTIMISATION TASK

Code of Conduct:

I B.Muni Chandini (192524283) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *B.Muni Chandini*

Q. Debugging Insertion Sort (Incorrect Index Placement)
The insertion sort code incorrectly places elements due to wrong index update. Identify the
logical error in shifting logic. Explain why elements are misplaced. Debug step-by-step.
Optimize code readability. Analyze time complexity and rewrite corrected version.
while j >= 0 and arr[j] > key:
arr[j] = arr[j-1] # ERROR
j -= 1

Q. Debugging Insertion Sort

1. Objective

The primary objective of this task is to identify, analyse and correct a logical error present in the shifting logic of the Insertion Sort algorithm. The given code incorrectly updates array indices while shifting elements, which results in misplaced values and an unsorted output.

Secondary objectives include:

- Explaining why the elements become misplaced due to the faulty index update.
- Debugging the algorithm step-by-step with a concrete example.
- Rewriting a clean, readable and correct version of Insertion Sort.
- Analysing the time complexity of the corrected algorithm.
- Suggesting minor optimisations for better readability and practical performance.

2. Problem Identification

2.1 Observed Issue

The following buggy fragment is given:

```
while j >= 0 and arr[j] > key:
    arr[j] = arr[j-1]        # ERROR
    j -= 1
```

When this code is executed, the array does not get sorted correctly. Elements are overwritten or shifted in the wrong direction, producing a corrupted array that is neither sorted nor a valid permutation of the original input.

2.2 Impact

- The sorting algorithm fails to produce a correctly ordered list.
- Data integrity is lost because values are overwritten incorrectly.
- Any downstream process that depends on sorted data (searching, reporting, analysis) receives wrong results.
- The bug is silent — it does not raise an exception, making it harder to detect without careful testing.

3. Root Cause Analysis

In the standard Insertion Sort algorithm, when a key is to be inserted into its correct position, all larger elements in the sorted portion must be shifted one position to the

right to create a vacant slot for the key.

The correct shifting statement is:

arr[j + 1] = arr[j]

However, the given code performs:

arr[j] = arr[j - 1]

This statement shifts elements to the

left instead of the right. Consequently:

- The element at index j is overwritten by the value at j−1.
- The original value at j is permanently lost.
- The vacant position needed for the key is never created in the correct place.
- After the loop ends, placing the key produces a corrupted array.

3.1 Why Elements Are Misplaced

Because the shift direction is reversed, each iteration destroys information rather than making space. The algorithm effectively loses values and places the key into an already occupied or incorrect index, resulting in duplicates, missing numbers, and an unsorted sequence.

4. Debugging Methodology

The following systematic steps were used to locate and confirm the bug:

4.1 Diagnostic Steps

1. Step 1: Read the standard Insertion Sort algorithm and compare the shifting logic with the given code.
2. Step 2: Identify that the assignment arr[j] = arr[j-1] moves data leftward instead of rightward.
3. Step 3: Manually dry-run the buggy code on a small array to observe the corruption.
4. Step 4: Dry-run the corrected statement arr[j+1] = arr[j] on the same input and verify correct behaviour.
5. Step 5: Confirm that after the while loop the key is placed at index j+1.

4.2 Concrete Dry-Run (Buggy Code)

Input array: [5, 2, 4]

First pass (i = 1, key = 2):

Initial: j = 0, arr = [5, 2, 4]

Condition 5 > 2 is true → execute arr[0] = arr[−1] (invalid / undefined behaviour)

Even if boundary is protected, the left-shift logic immediately destroys the sorted prefix.

Result after buggy execution: array becomes corrupted and is no longer a permutation of the original values.

4.3 Validation Process

- Tested the corrected algorithm on multiple inputs: already sorted, reverse sorted, random, and arrays with duplicates.
- Verified that the output is always a sorted permutation of the input.
- Compared results with Python's built-in sorted() function for confirmation.

5. Solution Implemented

The logical error is fixed by changing the shifting direction and ensuring the key is inserted at the correct vacant index.

5.1 Corrected Algorithm (Pseudocode)

```
InsertionSort(arr, n):
    for i ← 1 to n−1 do
        key ← arr[i]
        j ← i − 1
        while j ≥ 0 and arr[j] > key do
            arr[j + 1] ← arr[j]     // shift right (CORRECTED)
            j ← j − 1
        arr[j + 1] ← key            // insert key in vacant slot
```

5.2 Corrected Python Implementation (Readable Version)

```
def insertion_sort(arr):
    """Sort the list in-place using Insertion Sort."""
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        # Shift elements greater than key one position to the right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]     # CORRECTED LINE
            j -= 1
        arr[j + 1] = key            # Place key in the correct position
    return arr
```

5.3 Before vs After Comparison

Buggy Statement	Corrected Statement
arr[j] = arr[j-1]	**arr[j+1] = arr[j]**
Shifts elements LEFT	Shifts elements RIGHT
Destroys existing values	Creates space for the key
Produces corrupted array	Produces correctly sorted array

6. Optimization Techniques

Although Insertion Sort is inherently O(n²), several readability and micro-optimisations improve the practical quality of the code:

- **Clear variable naming:** Using 'key' and 'j' with descriptive comments makes the intention obvious.

- **Single responsibility:** The shifting loop only moves elements; key insertion is performed once after the loop.
- **Early exit opportunity:** When the array is already sorted, the inner while loop never executes, giving best-case O(n) behaviour.
- **In-place operation:** No extra array is allocated, keeping auxiliary space O(1).
- **Readability over micro-optimisation:** Avoided clever tricks that reduce clarity; the corrected version prioritises maintainability.

6.1 Optional Binary-Search Optimisation (Advanced)

The insertion position can be found using binary search in O(log n) comparisons. However, the shifting of elements still costs O(n) time, so the overall complexity remains $O(n^2)$. This optimisation reduces the number of comparisons but does not improve asymptotic time and slightly increases code complexity. It is therefore not applied in the final version.

7. Performance Evaluation

7.1 Time Complexity

Case	Comparisons / Moves	Time Complexity
Best Case (already sorted)	n−1 comparisons, 0 moves	**O(n)**
Average Case	~ $n^2/4$ operations	$\Theta(n^2)$
Worst Case (reverse sorted)	~ $n^2/2$ operations	$\Theta(n^2)$

7.2 Space Complexity

Auxiliary space = O(1) — the algorithm sorts in-place using only a constant amount of extra memory (the key variable and index j).

7.3 Before-and-After Behaviour

Metric	Buggy Version	Corrected Version
Correctness	Fails (corrupted output)	**Always correct**
Data Integrity	Values lost / duplicated	**Permutation preserved**
Best-case Time	Undefined / crashes	**O(n)**
Worst-case Time	N/A (incorrect)	$\Theta(n^2)$
Stability	Not applicable	**Stable**

8. Conclusion

The given Insertion Sort implementation contained a critical logical error in the element-shifting step: it moved values to the left (arr[j] = arr[j-1]) instead of to the right (arr[j+1] = arr[j]). This reversed shift destroyed existing data and prevented the algorithm from creating the vacant slot required for correct insertion of the key.

By correcting the index arithmetic and placing the key at position j+1 after the loop, the algorithm regains its classic behaviour: it produces a correctly sorted array while remaining in-place and stable.

The corrected version has the well-known time complexities — O(n) in the best case and $\Theta(n^2)$ in the average and worst cases — and uses only O(1) extra memory. Readability was improved through clear variable names and explanatory comments without altering asymptotic performance.